¿Y si ambos repartos leyeran datos contiguos?

En el experimento anterior, el cíclico equilibraba mejor el número de operaciones, pero tardaba más. Había una pista: incluso con un solo proceso, cuando ambos repartos poseen las mismas columnas, sus tiempos eran distintos. Antes de descartar el reparto cíclico, decidimos reducir el costo de acceso e indexación del cálculo local.

Hipótesis

Después de recibir la columna j, cada proceso copia los valores que necesita a un vector ordenado como sus columnas propias. Esperamos que reutilizar ese vector compense el costo de copiarlo, especialmente en el cíclico. La copia se incluye en el tiempo: no damos por hecho que sea beneficiosa.

El cambio en C

Sea g(c) la columna global correspondiente a la columna local c. En la versión original, el bucle interno consulta el vector difundido mediante ese índice global. En la nueva versión lo prepara una sola vez por etapa:

$$b_c = l_{g(c),\,j}, \qquad g(c) > j$$

$$a_{ic}^{\text{local}} \leftarrow a_{ic}^{\text{local}} - l_{ij} b_c$$

Momento	Versión original	Con vector local
Antes de actualizar	Se usa la columna difundida.	local_column[c] = column[g(c)]
Dentro del bucle	lij * column[g(c)]	lij * local_column[c]

Por ejemplo, con n=8 y P=3, el rango 1 cíclico posee las columnas globales 1, 4 y 7. En la etapa j=0 copia esos tres valores a posiciones locales consecutivas. Las filas que actualiza luego reutilizan esa copia.

Cuánto cuesta y qué se conserva

Cada proceso reserva tantos doubles adicionales como columnas propias: ocho bytes por columna local en este equipo. La memoria adicional máxima por proceso es:

$$M_{\text{extra}} \leq 8\left\lceil \frac{n}{P} \right\rceil \text{ bytes}$$

Durante toda la factorización se copian n(n−1)/2 valores entre todos los procesos. Es trabajo cuadrático adicional frente al cálculo cúbico de Cholesky; eso por sí solo no garantiza una mejora para tamaños pequeños.

No cambian los propietarios, las dependencias, el orden de las operaciones aritméticas ni las llamadas a MPI_Bcast. El bucle interno accede ahora a datos consecutivos en ambos repartos. La asignación de índices fuera de ese bucle todavía difiere.

Un experimento con cuatro variantes

Reparto	Lectura original	Vector local
Contiguo	Referencia anterior	Nueva variante
Cíclico	Referencia anterior	Nueva variante

Cómo evitamos mezclar campañas

Volvimos a medir las cuatro variantes en el mismo ejecutable y sobre las mismas matrices. Los cocientes de esta etapa usan únicamente tiempos nuevos; no dividimos tiempos actuales por los del PDF anterior. La generación y todas las factorizaciones están en C. Python valida los CSV y obtiene estadísticas y gráficos.

Configuración	Protocolo ejecutado
Entorno	WSL2 / Ubuntu 26.04, AMD Ryzen 5 2600, Open MPI 5.0.10, GCC 15.2.0; -O3, double, sin BLAS.
Tamaños y procesos	n=64,256,512,1024 y P=1,2,3,4. Un proceso por núcleo distinto según la topología expuesta por WSL.
Orden y repeticiones	Un calentamiento por variante; ocho repeticiones medidas. Cada variante ocupa cada posición dos veces. Se equilibran también las parejas consecutivas dentro de cada repetición.
Tiempo MPI	Máximo de las duraciones locales, con inicio coordinado por barrera. Incluye copia del vector, cálculo y difusión.
Fuera del cronómetro	Reserva de memoria, generación, distribución inicial, reunión, validación y arranque de MPI.
Referencia secuencial	Left, right original y right con vector: seis ejecuciones antes y seis después de MPI para cada n.

Qué reportamos

$$G_{\text{copia}} = \frac{\widetilde{T}_{\text{original}}}{\widetilde{T}_{\text{local}}}, \qquad S_{\text{util}} = \frac{\widetilde{T}_{\text{mejor secuencial}}}{\widetilde{T}_{\text{MPI}}}$$

Un cociente mayor que 1 indica una mejora. Usamos medianas y cuartiles inclusivos Q1-Q3. Las barras de dispersión no son intervalos de confianza; ocho repeticiones en una campaña no sustituyen repeticiones independientes en otros momentos.

Se conservaron 512 tiempos MPI, 144 tiempos secuenciales y un perfil separado de 48 registros por rango (n=1024, P=3, cuatro variantes y cuatro repeticiones). El perfil incorpora cronómetros internos y no se usa para calcular la aceleración principal.

Resultado: la copia ayuda sobre todo al cíclico

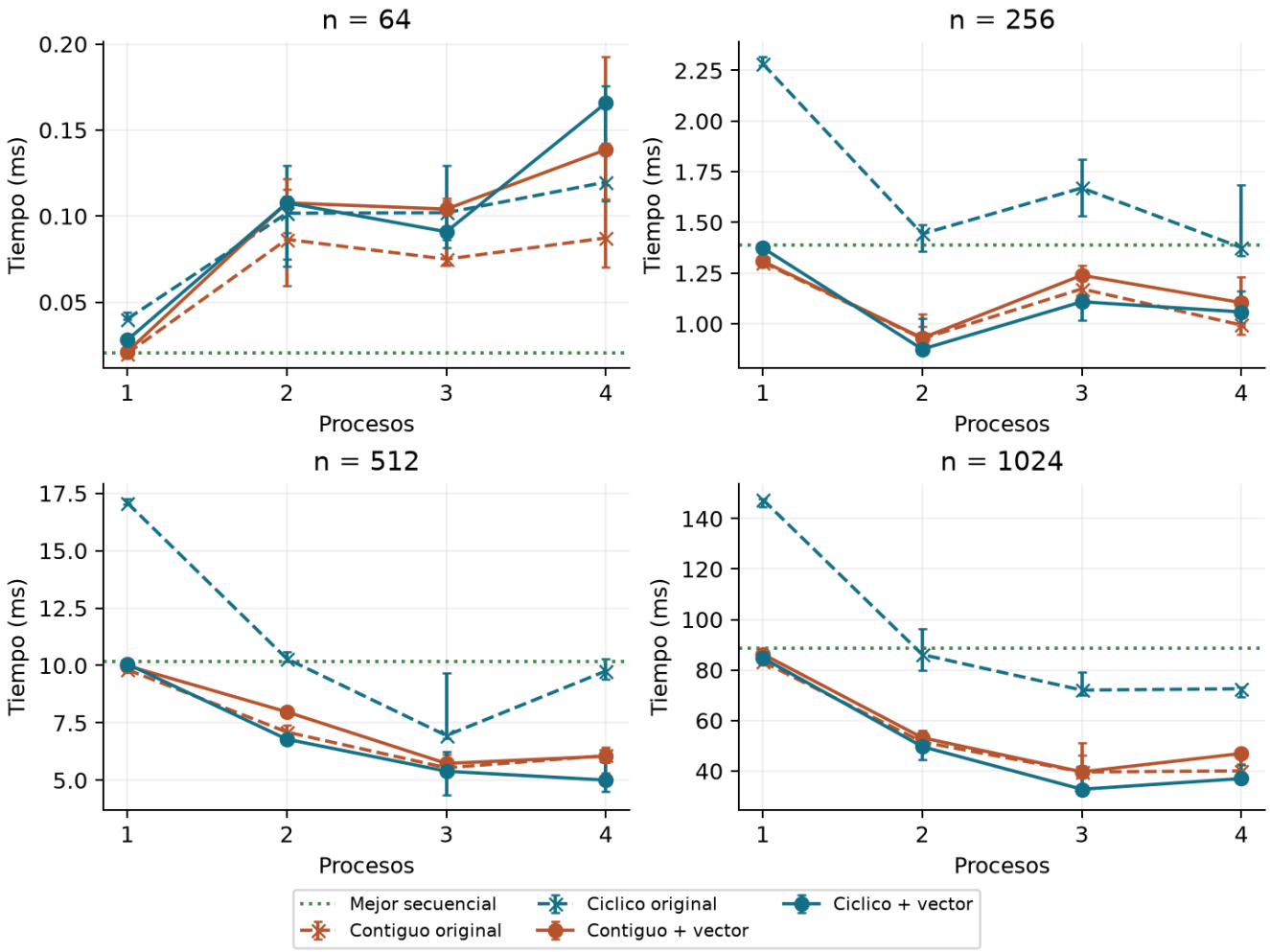


Figura 1. Medianas de ocho repeticiones; barras Q1-Q3. Líneas discontinuas: acceso original. Líneas continuas: vector local. La referencia secuencial es la mejor mediana de las tres variantes secuenciales de esta campaña. Cada panel tiene su propia escala.

La hipótesis se cumple con límites

n=64: todos los casos con P>1 son más lentos que el mejor secuencial; agregar una copia no resuelve la falta de trabajo útil.
n≥256: el cíclico con vector tiene menor mediana que el cíclico original en todos los P probados.
Contiguo: copiar no aporta una mejora sistemática; su lectura original ya era consecutiva y en varios casos la copia agrega costo.

Entre las nuevas variantes, el cíclico presenta menor mediana que el contiguo para P=2,3,4 y n≥256. Sin embargo, varias distribuciones de tiempos se superponen: esto respalda una elección provisional, no una superioridad universal ni una prueba de significación estadística.

El caso n=1024, explicado con números

P	Contiguo original	Contiguo + vector	Cíclico original	Cíclico + vector
1	83,28	86,33	147,55	84,81
2	51,49	53,17	86,11	49,64
3	39,62	39,66	72,04	32,77
4	40,03	46,93	72,57	36,99

Todos los tiempos están en milisegundos. Mediana del mejor secuencial: 88,62 ms, correspondiente a right con vector.

¿Se amortizó la copia?

Con P=3, el cíclico pasa de 72,04 ms a 32,77 ms: mejora 2,20 veces respecto de su versión original, incluyendo el tiempo de copiar. Frente al mejor secuencial, la aceleración útil es 2,70.

El control con un solo proceso

Antes de copiar, P=1 da 83,28 ms en contiguo y 147,55 ms en cíclico. Con el vector local, quedan en 86,33 y 84,81 ms, con cuartiles superpuestos. La gran diferencia previa se redujo: el reparto entre procesos no podía explicar esa diferencia inicial.

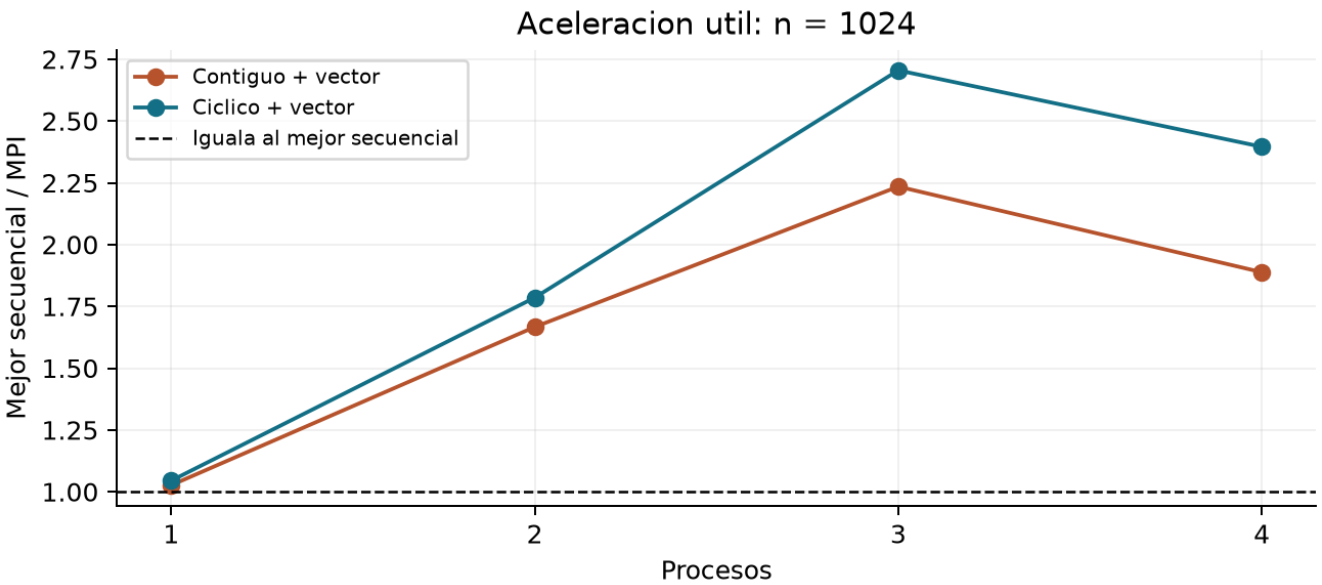


Figura 2. Aceleración útil de las nuevas variantes. Tres procesos cíclicos dieron la menor mediana para n=1024; sus cuartiles se superponen con los de cuatro procesos y con el contiguo de tres. Debemos repetir antes de afirmar cuál es la mejor configuración de forma estable.

Qué decidimos y cuál es la próxima pregunta

Qué podemos concluir

Conservar **right-looking MPI cíclico con vector local** como candidato principal está ahora respaldado por una mejora medida, no solo por el conteo de operaciones. También conservamos el contiguo original como referencia: la copia no lo mejora sistemáticamente. Para matrices pequeñas, mantenemos el secuencial como opción práctica.

El informe del compilador registra vectorización del nuevo bucle interno. El perfil separado muestra menor tiempo de cálculo local en el cíclico con vector para los tres rangos de n=1024, P=3. Es evidencia compatible con la mejora del acceso y la indexación; no cuantifica cuánto proviene de caché, instrucciones vectoriales u otros efectos.

El tiempo dentro de MPI_Bcast incluye esperas además de transferencia. El perfil tiene instrumentación y se ejecuta en otra pasada: sus tiempos no deben sustituir las medianas principales. No medimos comunicación entre nodos ni el tiempo completo de la aplicación.

Validación y trazabilidad

Las cuatro variantes pasaron los casos SPD, el factor conocido, repartos desiguales, P>n y fallos colectivos con P=1,2,3,4. También verificamos los límites del vector local y ejecutamos AddressSanitizer/UndefinedBehaviorSanitizer con P=4. Se mantuvo la adaptación de Open MPI descrita en la etapa 2 solo para los sanitizadores, con detección de fugas desactivada.

Las 512 observaciones MPI fueron aceptadas. El residuo máximo fue $1{,}2131\times10^{-15}$, inferior al límite 10^{-12}. La máxima diferencia registrada del factor frente a right secuencial fue cero en esta campaña. Se conservaron entradas del protocolo, afinidad, CSV y huellas de las fuentes.

¿En qué tamaño empieza a compensar esta versión?

Para responder mejor, el próximo paso razonable es probar tamaños intermedios entre 64 y 256, repetir campañas independientes y comparar pocos valores de P. Así evitamos elegir un umbral por una sola mediana. Después repetiremos el protocolo en el clúster, separando un nodo de varios nodos; los bloques y la planificación dinámica quedan para cuando los datos justifiquen su complejidad.

Archivo en desde_cero	Función
src/cholesky_mpi.c	Conserva el kernel original y añade cholesky_mpi_local.
src/compare_mpi_local.c	Compara las cuatro variantes y valida sus factores.
scripts/comparar_mpi_local.sh	Ejecuta la campaña reproducible.
scripts/analizar_mpi_local.py	Valida observaciones y genera estadísticas y gráficos.
resultados/mpi_local_20260921/	Datos, perfil, fuentes y comprobaciones.